Déploiement d'une IA Locale

Le but de ce TP est de déployer une solution d'intelligence artificielle locale sur une infrastructure virtualisée. L'objectif est d'installer et de configurer un grand modèle de langage (LLM) accessible via une interface web, sans recours à des services cloud externes, afin de garantir la confidentialité des données.

Sommaire :

1) Choix technologiques
2) Préparation de la machine virtuelle
3) Installation de Docker
4) Installation d'Ollama
5) Téléchargement du modèle Qwen3
6) Déploiement d'Open WebUI
7) Problèmes rencontrés et solutions apportées
8) Sources

1) Choix technologiques

Dans ce projet, je mets en place une infrastructure d'intelligence artificielle complètement locale. Aucune donnée ne transite par internet lors des échanges avec l'IA : tout s'exécute sur ma machine virtuelle Debian 12, hébergée dans Hyper-V.

L'architecture repose sur trois composants :

Ollama (https://ollama.com/ , Qu'est-ce qu'Ollama ? L'outil de gestion des modèles IA) comme moteur d'inférence IA : il gère les modèles LLM (Qu'est-ce qu'un grand modèle de langage (LLM) ? | Cloudflare) et une API REST.

Qwen3:4b (https://www.ecole.cube.fr/blog/qwen-lassistant-ia-multifonction-dalibaba) comme modèle de langage car il est récent (2025), léger (2,5 Go) et parfaitement c'est le plus sûr pour une machine disposant de 8 Go de RAM.

Open WebUI, une interface utilisateur qui s'installe facilement.

Je choisis de faire tourner Ollama directement sur le système Debian en tant que service systemd, et Open WebUI dans un conteneur Docker. Comme ça Ollama à un accès direct aux ressources machine (CPU, RAM), ce qui améliore les performances, et Open WebUI reste isolé dans son conteneur avec toutes ses dépendances.

2) Préparation de la machine virtuelle

Je crée une machine virtuelle sous Hyper-V avec 8 Go de RAM, 4 cœurs de CPU et 40 Go de disque. Je configure le réseau en mode pont (bridge) afin que la VM obtienne une adresse IP sur le même réseau que mon poste Windows, ce qui me permettra d'accéder à l'interface web depuis mon navigateur. Pour un résultat plus optimal il aurait fallu plus de RAM pour utiliser un modèle Qwen plus performant.

Une fois connecté à Debian, je commence par mettre à jour le système et installer les outils essentiels avec sudo apt update && sudo apt upgrade -y et sudo apt install -y curl wget git htop net-tools (curl : télécharger des scripts depuis internet, htop : moniteur CPU/RAM en temps réel, net-tools : outils réseau) pour finir je note l'adresse IP de la VM avec ip a. La commande me retourne trois interfaces : lo (loopback, 127.0.0.1), eth0 (ma carte réseau principale avec l'IP 10.10.255.132) et par la suite docker0 (interface virtuelle créée automatiquement par Docker montrant qu'il est bien installé). Je note l'adresse IP d'eth0 car c'est elle que j'utilise pour accéder à l'interface web.

3) Installation de Docker

J'installe Docker via le script officiel avec curl -fsSL https://get.docker.com | sh . Docker me permettra de déployer Open WebUI avec toutes ses dépendances en une seule commande, sans risquer de perturber le reste du projet.

Je vérifie l'installation avec docker --version et docker run hello-world et active le système au démarrage avec systemctl enable docker

4) Installation d'Ollama

J'installe Ollama, le moteur qui va charger et exécuter le modèle de langage. Ollama s'installe automatiquement comme un service systemd, ce qui signifie qu'il démarre automatiquement avec la machine.

J'installe Ollama via le script officiel : curl -fsSL https://ollama.com/install.sh | sh

Je vérifie que le service est actif : sudo systemctl status ollama

Je fais l'activation du démarrage automatique : systemctl enable ollama

Lors du premier démarrage, Ollama affiche un avertissement : "Warning: no NVIDIA/AMD GPU detected, Ollama will run in CPU-only mode". C'est normal car ma VM Hyper-V n'a pas accès à une carte graphique dédiée. Ollama fonctionne donc en mode CPU uniquement, ce qui est plus lent mais fonctionnel.

```
oot@ia:~# curl -fsSL https://ollama.com/install.sh | sh
>> Installing ollama to /usr/local
>> Downloading ollama-linux-amd64.tar.zst
######################################################################## 100.0%
>> Creating ollama user...
>> Adding ollama user to render group...
>> Adding ollama user to video group...
>> Adding current user to ollama group...
>> Creating ollama systemd service...
>> Enabling and starting ollama service...
reated symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd
>> The Ollama API is now available at 127.0.0.1:11434.
>> Install complete. Run "ollama" from the command line.
ARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.
oot@ia:~# systemctl status ollama
 ollama.service - Ollama Service
    Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: enabled)
    Active: active (running) since Wed 2026-03-11 10:53:26 CET; 2min 11s ago
  Main PID: 5170 (ollama)
     Tasks: 9 (limit: 9216)
    Memory: 40.4M
       CPU: 70ms
    CGroup: /system.slice/ollama.service
            └─5170 /usr/local/bin/ollama serve
```

5) Téléchargement du modèle Qwen3

Je dispose de 8 Go de RAM sur ma VM. Après déduction de la mémoire utilisée par Debian (~500 Mo) et Docker + Open WebUI (~500 Mo), il me reste environ 7 Go pour le modèle. Je choisis qwen3:4b qui nécessite environ 2,5 Go.

Je télécharge le modèle Qwen3 4b : ollama pull qwen3:4b

Je vérifie que le modèle est bien disponible : ollama list

Je test en mode conversation directe dans le terminal : ollama run qwen3:4b "Qu'est-ce qu'un VLAN ?"

Je test le mode thinking (le modèle affiche son raisonnement étape par étape) : ollama run qwen3:4b "/think Qu'est-ce qu'un VLAN ?"

```
 think I'll avoid diving into VLAN types (access, trunk) or VLAN stacking unless asked. Focus on core concept.

inal check: Am I using too much jargon? Let me see... "broadcast domain" is necessary but I'll say "traffic groups" first then define. "Switches" is fine since it's basic networking.

kay, drafting in my head: Start with "VLAN stands for Virtual Local Area Network..." then immediately give the analogy. Keep it under 150 words? User probably wants concise answer.

h! And emphasize that it's virtual - meaning it's a software configuration on hardware switches, not physical networks. That's crucial for understanding it's flexible.

starts writing response'
..done thinking.

 **VLAN** (Virtual Local Area Network) is a **logical segmentation** of a physical network. Think of it as a "virtual" network that groups devices together *even if they're physically connected to the
ame switch or cable*. Here's a simple breakdown:

## Key Points:
. **Why it exists**:
  In traditional networks, all devices on the same physical switch share the **same broadcast domain** (meaning traffic from one device can reach every other device). This causes security risks, network
ongestion, and management headaches.
  → **VLANs solve this by splitting the network into smaller, isolated broadcast domains**.

. **How it works**:
  - Devices are grouped into VLANs based on **security needs, department, role, or location** (e.g., "Finance VLAN," "HR VLAN").
  - All devices in a VLAN communicate *as if they're on their own physical network*, **without needing new cables**.
  - Switches (network hardware) use **VLAN tags** to route traffic between VLANs *without* physical changes.

. **Real-world example**:
  Imagine an office with 50 devices:
  - **Without VLANs**: All devices hear each other's traffic (e.g., HR emails flood the finance team).
  - **With VLANs**: Finance devices stay in their own VLAN → *only* finance devices see finance traffic. HR devices stay isolated → **no cross-department leaks**.

. **Key benefits**:
  - 🔒 **Security**: Limit exposure (e.g., prevent admin devices from seeing sensitive data).
  - ⚡ **Performance**: Reduce broadcast traffic (e.g., fewer "noise" packets = faster networks).
  - 🛠**Flexibility**: Move devices between VLANs without changing physical cables.
  - 📊 **Simpler management**: Group devices by function (e.g., "all IoT devices in VLAN 10").

## Simple Analogy:
 **VLANs are like "virtual rooms" in a building**.
 - All devices in Room A (VLAN 10) talk *only* to each other.
 - Devices in Room B (VLAN 20) are completely separate.
 - Even if everyone shares the same hallway (physical switch), the rooms stay isolated.

## Why it matters:
LANs are a **fundamental tool** for modern networks (used in schools, offices, cloud services). They let you build **secure, efficient networks** without expensive hardware changes. Without VLANs,
etworks would be chaotic and insecure.

*In a sentence**:
 *"A VLAN is a virtual network segment that groups devices together logically for better security, performance, and management — all without physical rewiring."*

f you're learning networking, start with VLANs — they're the foundation for understanding how networks stay clean and secure! 😊
(Need a deeper dive into VLAN types, tagging, or how to configure them
```

6) Déploiement d'Open WebUI

J'installe Open WebUI dans un conteneur Docker.

Lancement du conteneur Open WebUI :

```
docker run -d \
  --name open-webui \
  --network=host \
  -v open-webui:/app/backend/data \
  --restart always \
  ghcr.io/open-webui/open-webui:main
```

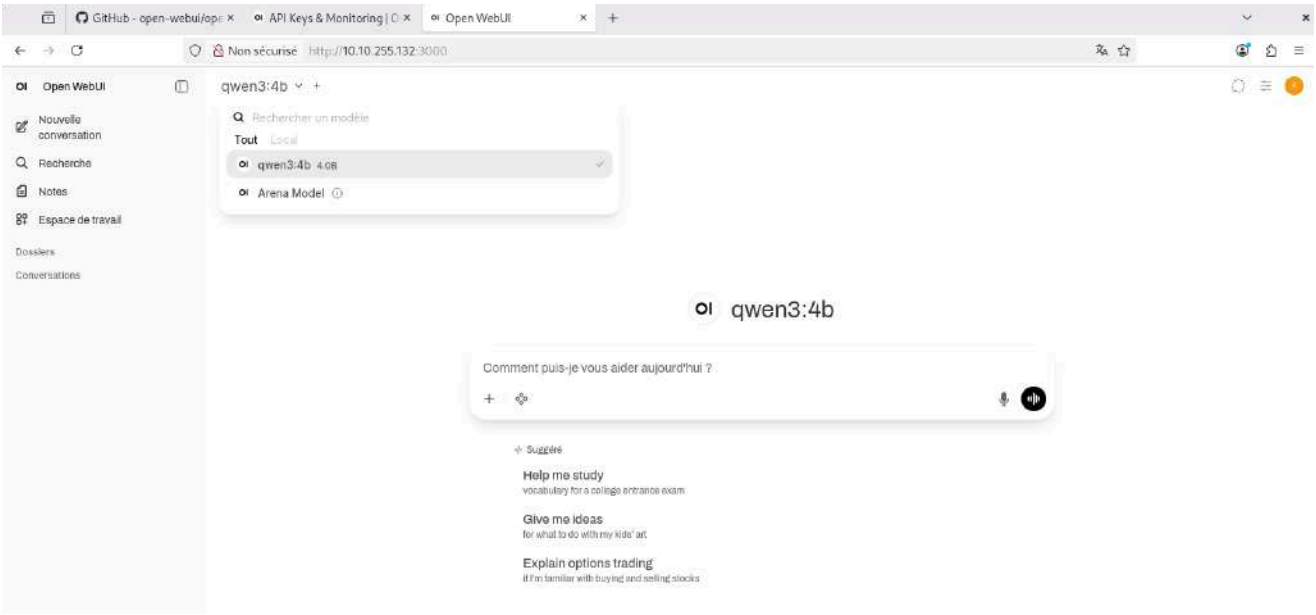
--network=host : le conteneur partage le réseau de la VM, nécessaire pour joindre Ollama sur localhost
-v open-webui:/app/backend/data : sauvegarde les conversations
--restart always : redémarre automatiquement si crash ou reboot

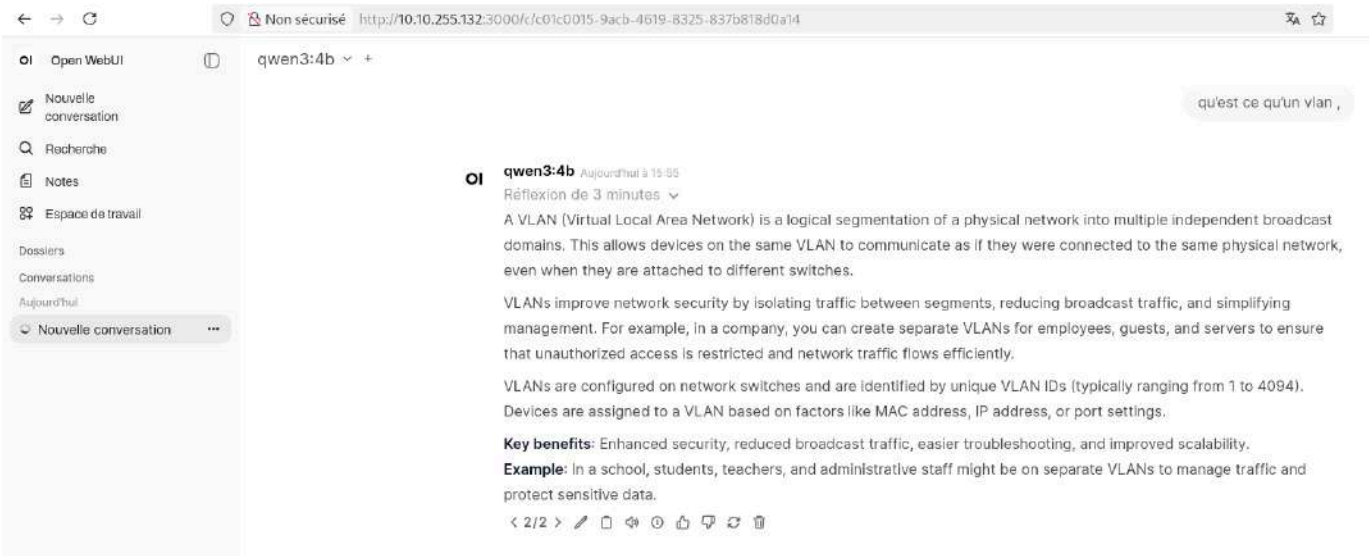
On peut également réaliser la commande sur une seule ligne :
`docker run -d --name open-webui --network=host -v open-webui:/app/backend/data --restart always ghcr.io/open-webui/open-webui:main`

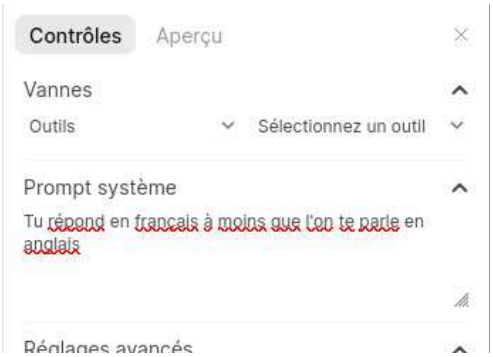
Pour avoir des logs de démarrage : `docker logs -f open-webui`

J'accède ensuite à l'interface depuis mon navigateur Windows à l'adresse http://10.10.255.132:3000 et je crée un compte administrateur (le premier compte créé obtient automatiquement les droits admin).

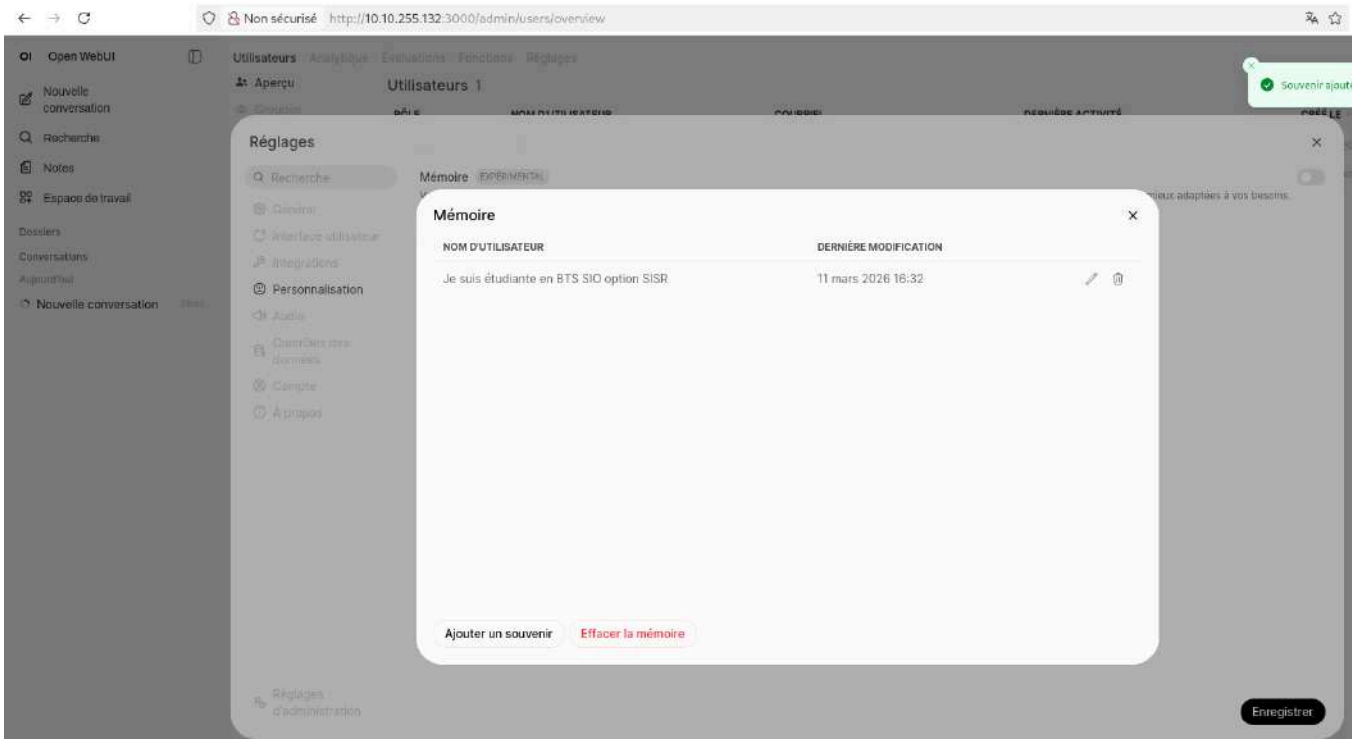
J’active ensuite mon modèle via le menu déroulant en haut à gauche et je navigue dans l’interface web pour personnaliser mon expérience. J’en profite pour re tester les réponses du modèle.


GitHub - open-webui/ope
API Keys & Monitoring | O
Open WebUI
Non sécurisé http://10.10.255.132:3000
Open WebUI
Nouvelle conversation
Recherche
Notes
Espace de travail
Dossiers
Conversations
qwen3:4b
Rechercher un modèle
Tout Local
qwen3:4b 4.0B
Arena Model
qwen3:4b
Comment puis-je vous aider aujourd'hui ?
Suggéré
Help me study
vocabulary for a college entrance exam
Give me ideas
for what to do with my kids' art
Explain options trading
if I'm familiar with buying and selling stocks


Non sécurisé http://10.10.255.132:3000/c/c01c0015-9acb-4619-8325-837b818d0a14
Open WebUI
Nouvelle conversation
Recherche
Notes
Espace de travail
Dossiers
Conversations
Aujourd'hui
Nouvelle conversation
qwen3:4b
qu'est ce qu'un vlan ,
qwen3:4b Aujourd'hui à 15:55
Réflexion de 3 minutes
A VLAN (Virtual Local Area Network) is a logical segmentation of a physical network into multiple independent broadcast domains. This allows devices on the same VLAN to communicate as if they were connected to the same physical network, even when they are attached to different switches.
VLANs improve network security by isolating traffic between segments, reducing broadcast traffic, and simplifying management. For example, in a company, you can create separate VLANs for employees, guests, and servers to ensure that unauthorized access is restricted and network traffic flows efficiently.
VLANs are configured on network switches and are identified by unique VLAN IDs (typically ranging from 1 to 4094). Devices are assigned to a VLAN based on factors like MAC address, IP address, or port settings.
Key benefits: Enhanced security, reduced broadcast traffic, easier troubleshooting, and improved scalability.
Example: In a school, students, teachers, and administrative staff might be on separate VLANs to manage traffic and protect sensitive data.
2/2


Contrôles
Aperçu
Vannes
Outils
Sélectionnez un outil
Prompt système
Tu répond en francais à moins que l'on te parle en anglais
Réglages avancés

Non sécurisé http://10.10.255.132:3000/c/2cbda114-dd92-407d-a8b8-f5839f15dcec
Open WebUI
Nouvelle conversation
Recherche
Notes
Espace de travail
Dossiers
Conversations
Aujourd'hui
Nouvelle conversation
qwen3:4b
qu'est ce qu'une base de donnée ,
qwen3:4b
Réflexion de 4 minutes
Une base de données est un ensemble structuré et organisé de données qui permet de stocker, récupérer et gérer efficacement des informations, tout en évitant les redondances et en assurant leur intégrité. Les bases de données relationnelles (comme MySQL ou PostgreSQL) utilisent un modèle tabulaire, tandis que les bases non relationnelles (comme MongoDB) offrent une flexibilité accrue. Elles sont largement utilisées dans les applications web, les applications mobiles et la gestion des données par les entreprises.


Non sécurisé http://10.10.255.132:3000/admin/users/overview
Open WebUI
Nouvelle conversation
Recherche
Notes
Espace de travail
Dossiers
Conversations
Aujourd'hui
Nouvelle conversation
Utilisateurs
Aperçu
Utilisateurs 1
Réglages
Recherche
Personnalisation
Mémoire
Mémoire
NOM D'UTILISATEUR
DERNIÈRE MODIFICATION
Je suis étudiante en BTS SIO option SISR
11 mars 2026 16:32
Ajouter un souvenir
Effacer la mémoire
Enregistrer

7) Problèmes rencontrés et solutions apportées

Problème : Aucun modèle détecté dans Open WebUI

Open WebUI se connecte mais n'affiche aucun modèle disponible, même après avoir téléchargé qwen3:4b. En fait, Ollama écoute uniquement sur 127.0.0.1 (loopback) par défaut et docker étant isolé dans le réseau, ses requêtes sont considérées comme externes et donc Ollama les refuse.

Je vérifie l'adresse sur laquelle écoute Ollama avec ss -tlnp | grep 11434
J'obtiens LISTEN 127.0.0.1:11434

Je vérifie que les modèles téléchargés par Ollama : ollama list
Je vois bien qwen3:4b dans la liste

Je force Ollama à écouter sur toutes les interfaces réseau en définissant OLLAMA_HOST=0.0.0.0 dans sa configuration systemd en créant le fichier de configuration override pour systemd : mkdir -p /etc/systemd/system/ollama.service.d/ et sudo tee /etc/systemd/system/ollama.service.d/override.conf << EOF

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
EOF
```

Je recharge la configuration systemd et redémarre Ollama :

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Je vérifie que Ollama écoute maintenant sur *:11434
ss -tlnp | grep 11434
Résultat : LISTEN *:11434

La valeur 0.0.0.0 signifie "écouter sur toutes les interfaces réseau". En passant de 127.0.0.1 à 0.0.0.0, Ollama accepte désormais les connexions venant de Docker, du réseau local et de partout.

```
root@ia:~# docker run -d --network=host -v open-webui:/app/backend/data -e OLLAMA_BASE_URL=http://127.0.0.1:11434 --name open-webui --restart always ghcr.io/open-webui/open-webui
:main
docker: Error response from daemon: Conflict. The container name "/open-webui" is already in use by container "03322313b653face4db547df853ec4c8478db0f5950ddbd090349fcb96da0d48".
You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
root@ia:~# ss -tlnp | grep 11434
LISTEN 0      4096       127.0.0.1:11434      0.0.0.0:*    users:(("ollama",pid=713,fd=3))
root@ia:~# journalctl -u ollama -f
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.765+01:00 level=INFO source=routes.go:1660 msg="Ollama cloud disabled: false"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.773+01:00 level=INFO source=images.go:477 msg="total blobs: 5"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.773+01:00 level=INFO source=images.go:484 msg="total unused blobs removed: 0"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.773+01:00 level=INFO source=routes.go:1713 msg="Listening on 127.0.0.1:11434 (version 0.17.7)"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.777+01:00 level=INFO source=runner.go:67 msg="discovering available GPUs..."
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.780+01:00 level=INFO source=server.go:430 msg="starting runner" cmd="/usr/local/bin/ollama runner --ollama-engine --port
 46627"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.876+01:00 level=INFO source=server.go:430 msg="starting runner" cmd="/usr/local/bin/ollama runner --ollama-engine --port
 45395"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.908+01:00 level=INFO source=runner.go:106 msg="experimental Vulkan support disabled.  To enable, set OLLAMA_VULKAN=1"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.908+01:00 level=INFO source=types.go:60 msg="inference compute" id=cpu library=cpu compute="" name=cpu description=cpu l
ibdirs=ollama driver="" pci_id="" type="" total="11.7 GiB" available="11.2 GiB"
mars 11 12:26:26 ia ollama[713]: time=2026-03-11T12:26:26.908+01:00 level=INFO source=routes.go:1763 msg="vram-based default context" total_vram="0 B" default_num_ctx=4096
sudo cat /etc/systemd/system/ollama.service.d/override.conf
^[[A^C
root@ia:~# mkdir -p /etc/systemd/system/ollama.service.d/
root@ia:~# sudo tee /etc/systemd/system/ollama.service.d/override.conf << EOF [Service]
> Environment="OLLAMA_HOST=0.0.0.0"
> EOF
Environment="OLLAMA_HOST=0.0.0.0"
root@ia:~# nano /etc/systemd/system/ollama.service.d/override.conf
root@ia:~# systemctl daemon-reload
root@ia:~# systemctl restart ollama
```

```
root@ia:~# sudo cat /etc/systemd/system/ollama.service.d/override.conf
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
root@ia:~# systemctl daemon-reload
root@ia:~# systemctl restart ollama
root@ia:~# sudo systemctl show ollama | grep -i environ
Environment=PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin OLLAMA_HOST=0.0.0.0
root@ia:~# ss -tlnp | grep 11434
LISTEN 0      4096               *:11434            *:*    users:(("ollama",pid=4702,fd=3))
```

Problème : Double installation d'Ollama

En cherchant une solution au premier problème, j'ai testé l'image Open WebUI avec Ollama intégré (ghcr.io/open-webui/open-webui:ollama), ce qui crée une deuxième instance d'Ollama dans le conteneur Docker, en plus de celle déjà installée sur Debian. Je voulais tout bien mettre au propre, pour cela je supprime le conteneur avec Ollama embarqué : docker stop open-webui puis docker rm open-webui . Je supprime aussi le conteneur hello-world inutile : docker rm gracious_babbage et relance Open WebUI sans Ollama embarqué avec la commande vu précedemment.

```
root@ia:~# docker ps
CONTAINER ID   IMAGE                                   COMMAND           CREATED        STATUS                    PORTS
          NAMES
03322313b653   ghcr.io/open-webui/open-webui:ollama   "bash start.sh"   26 hours ago   Up 3 minutes (healthy)    0.0.0.0:3000->8080/tcp, [::]:3000->8
080/tcp   open-webui
root@ia:~# docker ps -a
CONTAINER ID   IMAGE                                   COMMAND           CREATED        STATUS                     PORTS
          NAMES
03322313b653   ghcr.io/open-webui/open-webui:ollama   "bash start.sh"   26 hours ago   Up 4 minutes (healthy)     0.0.0.0:3000->8080/tcp, [::]:3000->
8080/tcp   open-webui
83e5c944f6a5   hello-world                             "/hello"          28 hours ago   Exited (0) 28 hours ago
          gracious_babbage
root@ia:~# ollama list
NAME        ID              SIZE      MODIFIED
qwen3:4b    359d7dd4bcda    2.5 GB    27 hours ago
root@ia:~# docker rm gracious_babbage
gracious_babbage
root@ia:~# docker stop open-webui
open-webui
root@ia:~# docker rm open-webui
open-webui
root@ia:~# docker run -d \
  --name open-webui \
  --network=host \
  -v open-webui:/app/backend/data \
  --restart always \
  ghcr.io/open-webui/open-webui:main
8eca362c5a04dd192d11533c03692923427bb6527da9896df16d3745f18da73f
root@ia:~# docker ps
CONTAINER ID   IMAGE                                   COMMAND           CREATED          STATUS                            PORTS     NAMES
8eca362c5a04   ghcr.io/open-webui/open-webui:main      "bash start.sh"   8 seconds ago    Up 7 seconds (health: starting)             open-webui
```

Problème : Après avoir supprimé et recréé le conteneur Open WebUI, j'obtiens une erreur 500 (Internal Server Error) en essayant d'accéder à http://10.10.255.132:8080.

En recréant le conteneur, j'utilise une commande avec l'option -p 3000:8080, ce qui signifie "rediriger le port 3000 de la machine hôte vers le port 8080 du conteneur". Le conteneur continue d'écouter en interne sur le port 8080, mais depuis l'extérieur il faut utiliser le port 3000. Open WebUI n'est donc plus accessible sur le port 8080 mais sur le port 3000. J'accède à l'interface via la bonne adresse : http://10.10.255.132:3000.

En remettant mon installation au propre, je reviens à l'option --network=host qui partage le réseau de la VM sans mapping de ports, rendant Open WebUI accessible sur son port natif 8080.

8) Sources

Ollama

Quick Start | Open WebUI

Qu'est-ce qu'Ollama ? L'outil de gestion des modèles IA

Qu'est-ce qu'un grand modèle de langage (LLM) ? | Cloudflare

https://claude.ai/

How to Set Up and Run Qwen3 Locally With Ollama | DataCamp

https://www.ecole.cube.fr/blog/qwen-lassistant-ia-multifonction-dalibaba